Informe de la Guía de Aprendizaje N° 4

UML, CONSTRUCTORES, LA REFERENCIA THIS Y MÉTODOS ESPECIALIZADOS EN JAVA

Estudiante: Marcos Riquelme Huahuiza Sarayasi
Institución: Instituto Superior Tecnológico de La Joya
Unidad didáctica: Taller de Lenguajes de Programación
Proyecto base: GameStore Escolar

1. Introducción

En esta sesión trabajé con el proyecto GameStore Escolar, que es una tienda donde los estudiantes juntan puntos y después los canjean. La idea era no lanzarse directo a programar, sino primero dibujar cómo iba a estar organizado el sistema con un diagrama de clases UML, y después pasar ese dibujo a código en Java. Me pareció buena la forma de trabajar porque cuando ya tienes el diagrama, el código sale casi solo y te equivocas menos.
En este informe explico lo que hice, cómo lo hice y qué aprendí, dejando los espacios para el diagrama, el código y las capturas de la ejecución.

2. Objetivo

Modelar las entidades principales de un sistema con diagramas de clases UML y programarlas en Java usando constructores, la referencia this y los métodos de acceso y modificación (getters y setters), además de toString().

3. Herramientas utilizadas

Usé el JDK (OpenJDK, versión 17 o superior) para compilar y ejecutar, y Visual Studio Code con la extensión Extension Pack for Java como editor. Para el diagrama usé la herramienta que indica la guía (Draw.io, Visual Paradigm o lápiz y papel): en este caso usamos lápiz y papel.

4. Marco teórico en pocas palabras

Diagrama de clases UML. Es un dibujo que muestra cómo está armado un sistema orientado a objetos. Cada clase es una caja dividida en tres partes: arriba el nombre, en el medio los atributos y abajo los métodos. El signo + significa público y el signo - significa privado. También se dibujan las relaciones entre las clases, con su multiplicidad (por ejemplo 1 y 0..*).
Constructor. Es un método especial que se ejecuta solo cuando creamos un objeto con new. Sirve para darle valores iniciales a sus atributos. Se llama igual que la clase y no devuelve nada, ni siquiera void.

La referencia this. Significa "yo mismo", o sea, el objeto actual. Se usa mucho cuando el parámetro tiene el mismo nombre que el atributo, para que Java sepa cuál es cuál.
Getters y setters. Los atributos son privados (encapsulamiento), así que desde afuera no se pueden tocar directamente. El getter (accessor) devuelve el valor y el setter (mutator) lo cambia. Así la clase controla quién cambia sus datos.
toString(). Es el método que convierte el objeto en un texto legible, con su estado actual. Sin él, al imprimir un objeto solo salen cosas raras como el nombre de la clase y un código.

5. Desarrollo

5.1. Trabajo 1: Diagrama de clases UML

La guía pedía modelar dos entidades mínimas del sistema:
Estudiante, con los atributos id (int), nombre (String) y puntos (int), y los métodos registrarPuntos() y consultarPuntos().
Transaccion, con los atributos idTransaccion (int), fecha (String) y costo (double), y el método procesarCanje().

Todos los atributos los puse como privados (signo -) y los métodos como públicos (signo +). Entre las dos clases dibujé una relación que dice "realiza": un estudiante puede realizar muchas transacciones (0..*), y cada transacción pertenece a un solo estudiante (1).

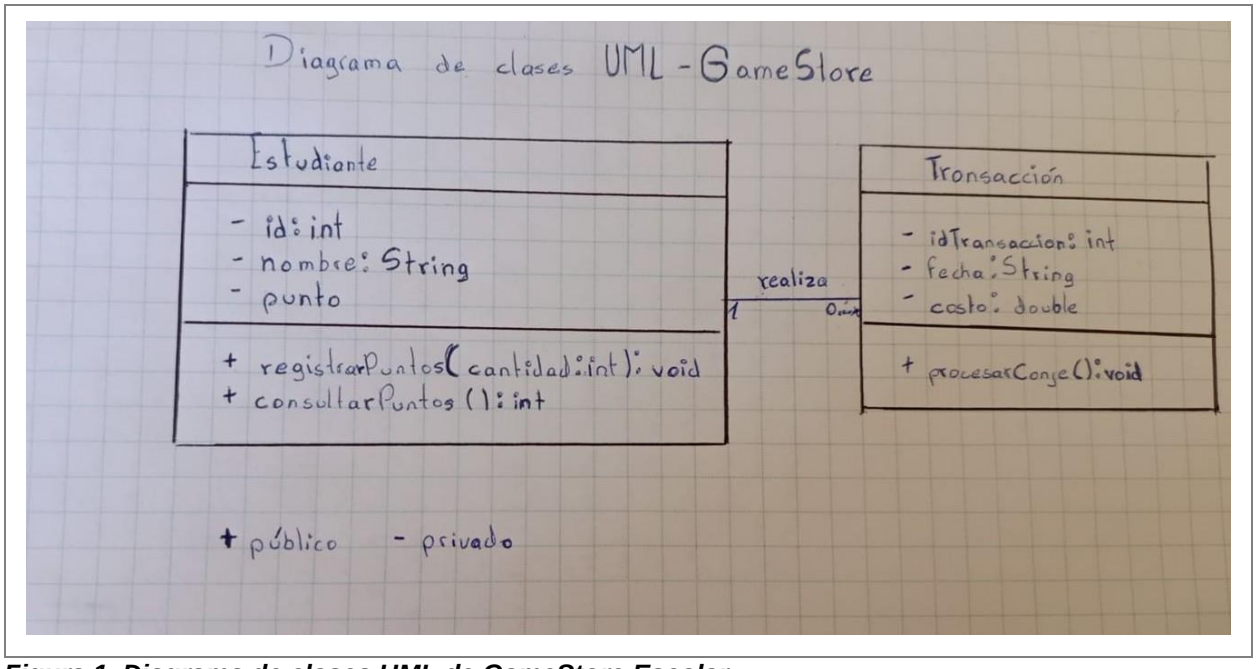


Figura 1. Diagrama de clases UML de GameStore Escolar.

Una aclaración: en el diagrama los métodos se llaman registrarPuntos() y consultarPuntos(), pero en el código Java usé setPuntos() y getPuntos(), porque es la convención estándar en Java. Cumplen el mismo rol, solo cambia el nombre.

5.2. Trabajo 2: Clase Estudiante.java

Después pasé el modelo a código. En la clase Estudiante hice lo siguiente:

Declaré los tres atributos como private, para aplicar encapsulamiento.
Escribí un constructor por defecto (vacío), por si se necesita crear un estudiante sin datos todavía.
Escribí un constructor con parámetros que recibe id, nombre y puntos, y los guarda con this.id = id;, this.nombre = nombre; y this.puntos = puntos;.
Creé un getter y un setter por cada atributo. Cada getter lleva su return y cada setter usa this para guardar el valor.
Sobrescribí toString() con @Override para que muestre el estado del objeto en una sola línea.

```java
public class Estudiante {
    // Atributos privados (Encapsulamiento)
    private int id;
    private String nombre;
    private int puntos;

    // Constructor por defecto
    public Estudiante() {
    }

    // Constructor parametrizado utilizando la referencia 'this'
    public Estudiante(int id, String nombre, int puntos) {
        this.id = id;
        this.nombre = nombre;
        this.puntos = puntos;
    }

    // Accessors (Getters) y Mutators (Setters)
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getPuntos() {
        return puntos;
    }
```

```
    public void setPuntos(int puntos) {
        this.puntos = puntos;
    }

    // Método especializado toString
    @Override
    public String toString() {
        return "Estudiante [ID=" + id + ", Nombre=" + nombre + ", Puntos="
+ puntos + "]";
    }
}
```

Código 1. Clase Estudiante.

5.3. Clase de ejecución: PrincipalEstudiante.java

Para probar que todo funciona hice una clase aparte, con el método main. Ahí creo un objeto alumno1 con el constructor parametrizado (id 1, nombre "Carlos Mendoza" y 450 puntos). Luego uso setPuntos(500) para actualizar los puntos, e imprimo el objeto con toString() y el nombre con getNombre().

```
public class PrincipalEstudiante {
    public static void main(String[] args) {
        // Creación de objeto usando el constructor parametrizado
        Estudiante alumno1 = new Estudiante(1, "Carlos Mendoza", 450);

        // Uso de mutators para actualizar datos
        alumno1.setPuntos(500);

        // Mostrar datos usando toString y accessors
        System.out.println(alumno1.toString());
        System.out.println("Nombre consultado con Getter: " +
alumno1.getNombre());
    }
}
```

ºCódigo 2. Clase PrincipalEstudiante.

5.4. Compilación y ejecución

Desde la terminal de VS Code, ubicado en la carpeta del proyecto, compilé las dos clases y luego ejecuté la principal:

```
javac Estudiante.java PrincipalEstudiante.java
java PrincipalEstudiante
```

La salida esperada en consola es:

```
Estudiante [ID=1, Nombre=Carlos Mendoza, Puntos=500]
Nombre consultado con Getter: Carlos Mendoza
```

Se ve que los puntos aparecen en 500 y no en 450, que es la prueba de que el setter funcionó.

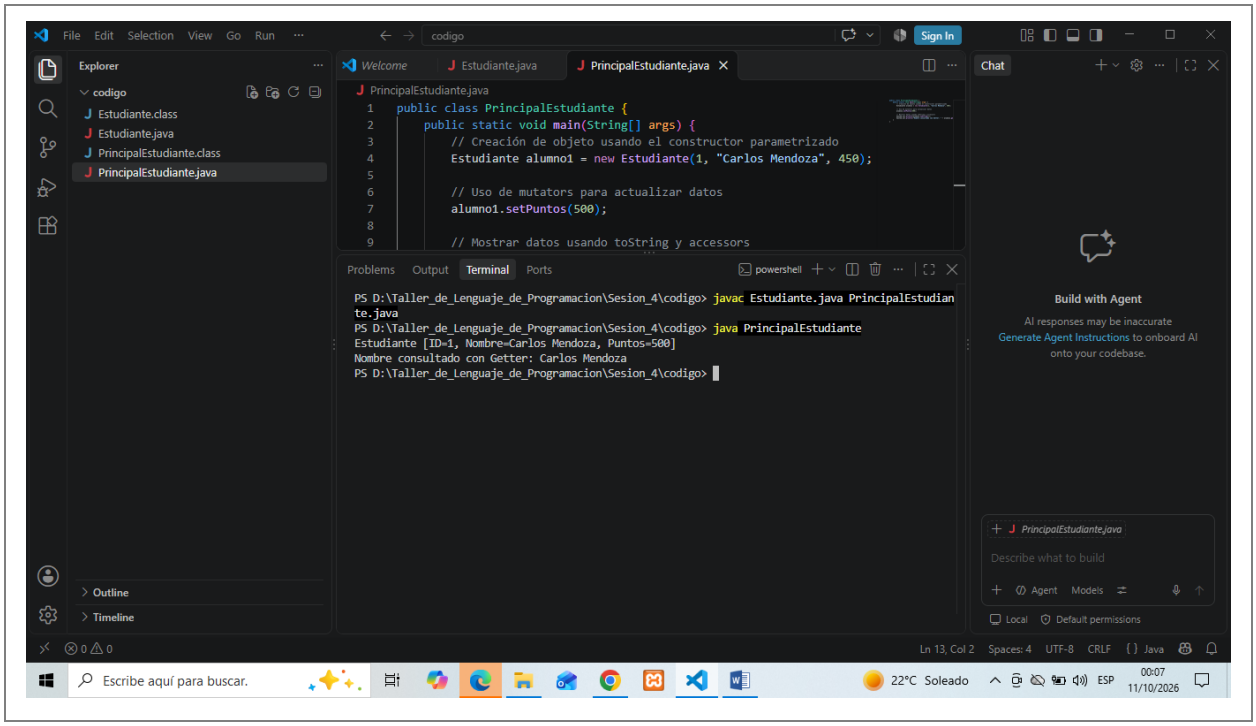


Figura 2. Compilación y salida del programa en la terminal.

6. Sustentación conceptual: ¿para qué sirve this en un constructor?

Dentro de un constructor, this hace referencia al objeto que se está creando en ese momento. Su utilidad principal es diferenciar el atributo de la clase del parámetro cuando los dos se llaman igual. Por ejemplo, en this.puntos = puntos; lo de la izquierda es el atributo del objeto y lo de la derecha es el parámetro que llegó al constructor.

Si me olvido del this y escribo solamente puntos = puntos;, Java entiende que le estoy asignando el parámetro a sí mismo, y el atributo nunca recibe el valor. En el caso de puntos, que es un int, se quedaría con su valor por defecto, que es 0. Lo bueno es que el programa compila igual, entonces el error no avisa y uno se da cuenta recién cuando el resultado sale mal.

7. Errores comunes y cómo los evité

Olvidar el this. Cuando el parámetro y el atributo se llaman igual, siempre escribí `this.atributo = atributo;`.

Atributos públicos. Los dejé todos private y el acceso lo hice solo con getters y setters.

Getter sin return. Revisé que cada getter devolviera su atributo con `return`.

8. Conclusiones

Con esta sesión entendí que modelar primero ayuda bastante: el diagrama UML me mostró qué clases, atributos y métodos necesitaba antes de escribir una sola línea de código. También quedó claro para qué sirve cada cosa: el constructor inicializa el objeto, this evita confusiones entre atributo y parámetro, los getters y setters protegen los datos, y toString() permite ver el estado del objeto de forma sencilla. Todo esto lo voy a seguir usando en las siguientes clases del proyecto GameStore Escolar, por ejemplo cuando programe la clase Transaccion.

9. Bibliografía

Guía de Aprendizaje N° 4: Diagramas UML, Constructores, la Referencia this y Métodos Especializados. Taller de Lenguajes de Programación.
Documentación oficial de Java (Oracle): https://docs.oracle.com/javase/tutorial/java/javaOO/constructors.html